

Design and Implementation

Monday, October 13, 2025 6:03 PM

Design and Implementation

- **Software Design** is the stage where the **conceptual or logical model** (idea-level plan) is turned into a **physical model** (real structure of code and components).
- In simple terms, it's about **planning how the system will actually work** — what components it will have, how they will interact, and how the goals will be achieved.
- The designer specifies the **software structure**, using components like classes, interfaces, modules, and their relationships.

Implementation

- **Implementation** means **translating the design into actual code** — turning the blueprint into a working software product.
- It's done systematically so that the code matches the design model and performs correctly.
- Example: If in design we planned a "Login" class with methods `authenticate()` and `logout()`, implementation is where we actually write those methods in a programming language (like Java, Python, etc.).

Object-Oriented Design using UML

- **Object-Oriented Design (OOD)** is a design approach where the system is built from **objects** — each representing a real-world entity (like a "User," "Product," or "Order").
- Each object has:
 - **Attributes (state)** — data that describe the object.
 - **Operations (methods)** — actions the object can perform.
- OOD focuses on **encapsulation** — meaning each object manages its own data and only interacts with others through defined interfaces.
- This approach makes software **easier to maintain, modify, and extend** compared to traditional (functional) design methods.

Common Activities in Object-Oriented Design

1. **Define the context and modes of use of the system**
 - Understand how the system will interact with users or other systems.
 - Example: In an online shopping system — who uses it? (Customer, Admin, Seller).
2. **Design the system architecture**
 - Break the system into major components (layers, modules).
 - Example: Presentation layer (UI), Business layer (logic), and Data layer (database).
3. **Identify the principal system objects**
 - Find the main entities of the system.
 - Example: User, Product, Cart, Payment, Order.
4. **Develop design models**
 - Use **UML diagrams** (like class, sequence, use case diagrams) to visually represent how objects and components interact.
5. **Specify object interfaces**
 - Define what methods or operations each class will expose to others.
 - Example: The Cart class might have methods like `addItem()`, `removeItem()`, and `calculateTotal()`.

System Context and Interactions

When we design a software system, it doesn't exist alone — it interacts with **people, devices, and other systems**.

So before we start designing the internal structure, we must first understand:

- **What is inside the system (its boundary)**
- **What is outside the system (external environment)**
- **How they communicate with each other**

This understanding helps decide:

- What functions the system itself will perform
- What will be handled by external systems

If we don't define the system boundary, we might include unnecessary features or miss some important external connections.

For example:

In an **ATM system**, the ATM software must interact with:

- Bank's main server (for verifying account & balance)
- SMS servers (for sending alerts to users)

But hardware functions like *printing receipts* or *dispensing cash* are handled by ATM hardware systems — not by the software itself.

(already discussed in previous chapters)

System Context Model

- It is a **structural model** — it shows the **other systems, actors, or devices** that exist around the main system.
- It helps visualize **what the system interacts with** and **how data flows** between them.

Interaction Model

- It is a **dynamic model** — it shows **how the system and environment interact over time** as actions occur.
- It focuses on **sequence and behavior**, not just structure.
- Example: When a customer inserts a card, the ATM system verifies it with the bank server, then allows withdrawal, and finally sends a notification through the SMS server.

Use Case Model (previously discussed)

A **use case model** shows **how external entities (actors)** interact with the system to achieve a **specific goal or function**.

Symbols in Use Case Diagram:

Symbol	Meaning
Actor (stick figure)	External user or system interacting with the system. (e.g., Customer, Bank Server, E-Sewa)
Use Case (oval)	Function or service performed by the system. (e.g., Withdraw Cash, Check Balance)
Line (association)	Shows communication between actor and use case.
System Boundary (rectangle)	Defines the scope of the system — what's inside and what's outside.

Purposes of a Use Case Diagram

- Understand **how the system will be used** by different users or systems.
- Helps in **requirement analysis** — ensures all necessary functions are included.
- Gives an **outside view** (user's perspective) of the system.
- Identifies **external and internal factors** influencing the system.

Example: ATM System Use Case Diagram

Actors:

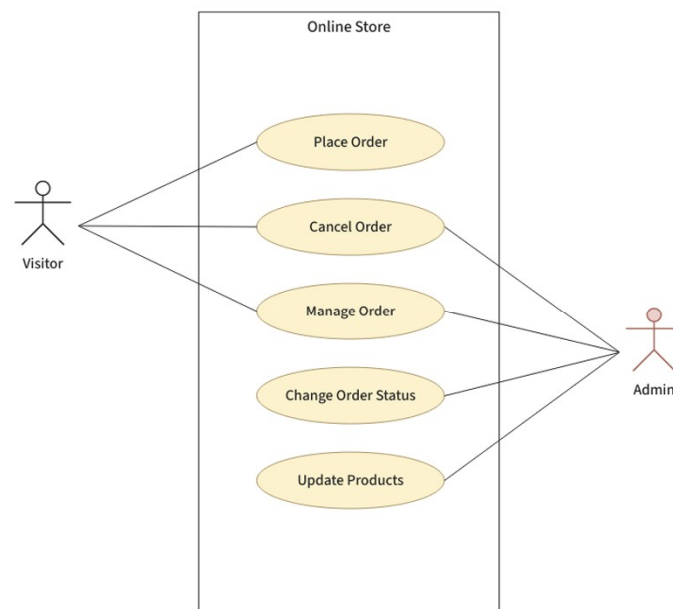
- Account Holder (Customer)
- CEO (Admin)
- e-Sewa API (External system)
- SMS Server (Notification service)

Use Cases:

- Make deposit
- Withdraw funds
- Change PIN/password
- Create account
- Verify documents
- Withdraw cash using ATM

System Boundary: “ATM System”**Textual Flow Example:**

1. Account Holder selects “Withdraw Cash”.
2. ATM requests verification from Bank Server.
3. If valid, transaction proceeds and amount is dispensed.
4. ATM sends transaction data to SMS Server → user receives notification.



Architectural Design

After defining the system context and use cases, the next step is **Architectural Design** —
→ breaking the system into **subsystems** (like database, interface, logic layers) and
→ defining **relationships** between them using architectural patterns (like layered, client-server, or repository).

Object Class Identification

- This is part of **Object-Oriented Design (OOD)**.
- It means finding the **main objects (classes)** that represent entities in your system.
- Example for ATM:
 - **Classes** → Customer, Account, Transaction, ATM, BankServer.
- There's no fixed rule — it depends on **experience and understanding** of the domain.
- It's an **iterative process** (refined multiple times).

Design Patterns

Design patterns are **standard solutions to common problems** in software design. Each pattern provides a **reusable blueprint** that can be adapted to fit different situations. They help make code **easier to understand, modify, and maintain**.

Design patterns act as **best practices** — instead of reinventing the wheel, developers can follow these tried and tested methods. They also create a **common language** among programmers, making team communication smoother.

Using design patterns helps to:

- Speed up development using **proven solutions**.
- Improve **code quality** and **readability**.
- Avoid **common design errors**.

Example:

In an **online shopping system**, you can use the **Singleton Pattern** to ensure that only one instance of the **Shopping Cart** exists per user.

This prevents multiple carts from being created for the same user and keeps data consistent.

Main Types of Design Patterns:

1. **Creational Patterns:** Deal with object creation.
Examples: Singleton, Factory Method, Builder.
2. **Structural Patterns:** Deal with object composition and relationships.
Examples: Adapter, Composite, Decorator.
3. **Behavioral Patterns:** Deal with communication between objects.
Examples: Observer, Strategy, Command.

Implementation Issues

Implementation issues are practical challenges and considerations faced during the process of turning software design into a working system. Some key issues include:

1. Reuse

Modern software development emphasizes **reusing existing components** rather than building everything from scratch. This reduces development time and cost while increasing reliability.

- **Object Level:** Reusing individual classes or objects from existing libraries.
Example: Using built-in Math or Date classes in Java instead of writing custom code for date handling or mathematical operations.
- **Component Level:** Reusing a collection of related objects or modules that provide a specific functionality.
Example: Integrating a pre-built **login module** or **payment gateway** (like PayPal API or eSewa SDK) into an online shopping system.
- **System Level:** Reusing an entire application or system and customizing it for new requirements.
Example: Adapting open-source platforms like **Moodle** for e-learning or **WordPress** for building a custom website.

2. Configuration Management

Configuration management is the process of **tracking and controlling changes** in software during development. It ensures that every team member works on the correct version of the code and that changes are properly documented.

Example:

Using tools like **Git** or **GitHub**, developers can:

- Store multiple versions of a project.
- Merge changes from different team members.
- Track who made each modification.

This helps maintain consistency and prevents version conflicts.

3. Host–Target Development

This refers to developing software on one system (the **host**) but running or testing it on another (the **target**).

- The **host** is the computer used for writing, compiling, and debugging code.
- The **target** is the device or environment where the final software runs.

Example:

Developing an **Android app** on a computer using **Android Studio (host)** and then testing it on a **smartphone (target)**.

Open Source Development

Open Source Development refers to a software development approach where the **source code is made publicly available**, allowing anyone to **inspect, modify, and share** it. This promotes **collaboration, transparency, and innovation** within the developer community.

Open-source projects often evolve faster because developers around the world contribute new features, fix bugs, and improve performance. However, contributions must follow specific **licenses** (like **GPL**, **MIT**, or **Apache License**) that define how the software can be used or redistributed.

Examples of Open-Source Software:

- **Linux** – an open-source operating system.
- **Apache Server** – a popular web server software.
- **MySQL** – a database management system.
- **Java** – an open-source programming language and platform.

Example:

If a developer discovers a bug in **MySQL**, they can fix it, test it, and submit their updated code to the open-source community. After review, the fix may be included in the next release — helping everyone who uses the software.